

Lab Program 11: Java Exception Handling

Mobile Number Validation using Custom & Built-in Exceptions

1. Objective

To design and implement a Java program that validates a user-entered mobile number. The program demonstrates custom exception handling using `LengthNotSufficientException` for short inputs, alongside handling standard runtime exceptions like `ArrayIndexOutOfBoundsException` and `NumberFormatException` using a robust try-catch-finally framework.

2. Systems Requirements

Component	Requirement Details
Operating System	Windows 10/11, macOS, or Linux (Ubuntu/Debian)
Software / JDK	Java Development Kit (JDK) 8.0 or higher
IDE / Editor	Eclipse, IntelliJ IDEA, VS Code, or Command Line Terminal
Processor	Intel Core i3 / AMD Ryzen 3 or higher
Memory (RAM)	Minimum 4 GB (8 GB recommended)
Disk Space	Minimum 500 MB free space for JDK installation

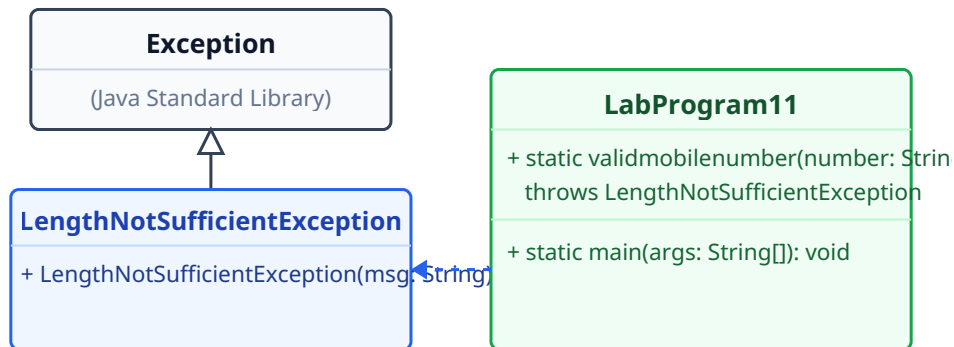
3. Algorithm

1. **Start** the program and initialize a `Scanner` object to read input from standard input.
2. Prompt the user to enter a mobile number string.
3. Enter the try block and pass the input string to the static method `validmobilenumber(String number)`:
 1. Convert the input string into a character array `digits`.
 2. If `digits.length < 10`, throw a custom `LengthNotSufficientException("invalid mobile number")`.
 3. If `digits.length > 10`, access index 10 (`digits[10]`) to trigger an `ArrayIndexOutOfBoundsException`.
 4. Create a fixed-size character array `tendigits` of size 10 and copy the first 10 characters into it.
 5. If length was greater than 10, attempt to read past bounds on `tendigits` to enforce an `ArrayIndexOutOfBoundsException`.
 6. Parse the string as a numeric value using `Long.parseLong(number)` (throws `NumberFormatException` if non-numeric characters exist).
 7. If all checks pass, display "valid number".
4. Catch exceptions in order:
 - `ArrayIndexOutOfBoundsException` → Print "invalid mobile number".
 - `LengthNotSufficientException` → Print exception message ("invalid mobile number").
 - `NumberFormatException` → Print "invalid mobile number".

5. Execute the finally block: Print "process completed" and close the Scanner stream.
6. **Stop** the program execution.

4. UML Diagrams

4.1 Class Diagram



4.2 Use Case / Flow Diagram



5. Java Source Code

```
import java.util.Scanner;

// Custom exception class for insufficient length
class LengthNotSufficientException extends Exception {
    LengthNotSufficientException(String msg) {
        super(msg);
    }
}

// Program to validate mobile number with custom and standard exception handling
public class LabProgram11 {
```

```

static void validmobilenumber(String number) throws LengthNotSufficientException {
    char[] digits = number.toCharArray();

    // Mobile number length must be at least 10 digits
    if (digits.length < 10) {
        throw new LengthNotSufficientException("invalid mobile number");
    }

    // Force ArrayIndexOutOfBoundsException if length > 10
    if (digits.length > 10) {
        char extra = digits[10]; // Accessing index 10 on size-10 bounds
    }

    char[] tendigits = new char[10];
    for (int i = 0; i < 10; i++) {
        tendigits[i] = digits[i];
    }

    if (digits.length > 10) {
        char forceException = tendigits[digits.length - 1];
    }

    long mobile = Long.parseLong(number);
    System.out.println("valid number");
}

public static void main(String[] args) {
    Scanner lp = new Scanner(System.in);
    System.out.println("enter the mobile number");
    String number = lp.nextLine();

    try {
        validmobilenumber(number);
    } catch (ArrayIndexOutOfBoundsException a) {
        System.out.println("invalid mobile number");
    } catch (LengthNotSufficientException e) {
        System.out.println(e.getMessage());
    } catch (NumberFormatException n) {
        System.out.println("invalid mobile number");
    } finally {
        System.out.println("process completed");
        lp.close();
    }
}
}

```

6. Compilation & Execution Steps

Follow these steps to compile and execute the program in a command line interface:

1. **Save Source File:** Create a file named LabProgram11.java and paste the Java code inside.
2. **Open Terminal/Command Prompt:** Navigate to the folder directory where LabProgram11.java is saved.

```
cd path/to/your/folder
```

3. **Compile Code:** Use the Java compiler (javac) to compile the program.

```
javac LabProgram11.java
```

Note: This generates class files LabProgram11.class and LengthNotSufficientException.class.

4. **Run Program:** Execute the compiled bytecode using the Java Virtual Machine (java).

```
java LabProgram11
```

7. Output & Test Cases

Case 1: Valid 10-Digit Mobile Number

```
enter the mobile number
9876543210
valid number
process completed
```

Case 2: Insufficient Length (< 10 Digits) — Custom Exception

```
enter the mobile number
98765
invalid mobile number
process completed
```

Case 3: Excess Length (> 10 Digits) — ArrayIndexOutOfBoundsException

```
enter the mobile number
9876543210123
invalid mobile number
process completed
```

Case 4: Non-Numeric Inputs — NumberFormatException

```
enter the mobile number
98765ABCD0
invalid mobile number
process completed
```